

QUESTION 1:

Step 1: Try $D_1 = 1$ (Morning). Valid so far.

Step 2: Try $D_2 = 1$ (Morning). **Backtrack!** Violates $D_1 \neq D_2$.

Step 3: Try $D_2 = 2$ (Afternoon). Valid so far ($D_1 \neq D_2$).

Step 4: Try $D_3 = 1$ (Morning). **Backtrack!** Violates $D_3 \neq 1$.

Step 5: Try $D_3 = 2$ (Afternoon). **Backtrack!** Violates $D_2 \neq D_3$.

Step 6: Try $D_3 = 3$ (Night).

- Constraint Check:
 1. $D_1 = 1 \neq 3$ ✓
 2. $D_2 = 2 < D_3 = 3$ ✓
 3. All variables distinct (1, 2, 3) ✓
 4. $D_3 = 3 \neq 1$ ✓
- **Goal State Reached!**

Doctor, Assigned Shift

Doctor 1 (D1), Morning

Doctor 2 (D2), Afternoon

Doctor 3 (D3), Night

Backtracking Search for Doctor Shift Scheduling Problem

SHIFTS = {1: "Morning", 2: "Afternoon", 3: "Night"}

def is_valid(assignment):

"""Checks whether the current partial or complete assignment satisfies all constraints."""

d1 = assignment.get("D1")

d2 = assignment.get("D2")

d3 = assignment.get("D3")

Constraint i: D1 cannot work Night shift

if d1 == 3:

return False

Constraint iv: D3 cannot work Morning shift

if d3 == 1:

return False

Constraint ii: D2 must work before D3 (Morning < Afternoon < Night)

if d2 is not None and d3 is not None:

if not (d2 < d3):

return False

```

# Constraint iii & v: All assigned shifts must be unique (Only one doctor per shift)
assigned_values = [v for v in assignment.values() if v is not None]
if len(assigned_values) != len(set(assigned_values)):
    return False

```

```

return True

```

```

def backtrack(assignment, variables, domains, step_counter):
    # Base Case: All variables are assigned
    if len(assignment) == len(variables):
        print(f"\n[GOAL REACHED] Valid Assignment Found: {assignment}")
        return assignment

    # Select unassigned variable
    unassigned = [v for v in variables if v not in assignment]
    var = unassigned[0]

    for val in domains:
        step_counter[0] += 1
        print(f"Step {step_counter[0]}: Trying {var} = {SHIFTS[val]} ({val})")

        assignment[var] = val

        if is_valid(assignment):
            print(f" -> Valid partial assignment: {assignment}")
            result = backtrack(assignment, variables, domains, step_counter)
            if result is not None:
                return result
        else:
            print(f" -> [PRUNED / BACKTRACK] Constraint violation at {assignment}")

    # Undo assignment (Backtrack)
    del assignment[var]

    return None

```

```

if __name__ == "__main__":
    variables = ["D1", "D2", "D3"]
    domains = [1, 2, 3] # 1: Morning, 2: Afternoon, 3: Night

    print("--- Starting Backtracking Search ---\n")

```

```

step_counter = [0]
solution = backtrack({}, variables, domains, step_counter)

print("\n" + "=" * 35)
print("    FINAL VALID SCHEDULE    ")
print("=" * 35)
if solution:
    for doc, shift in solution.items():
        print(f" {doc} -> {SHIFTS[shift]}")
else:
    print("No solution found.")
print("=" * 35)

```

--- Starting Backtracking Search ---

```

Step 1: Trying D1 = Morning (1)
-> Valid partial assignment: {'D1': 1}
Step 2: Trying D2 = Morning (1)
-> [PRUNED / BACKTRACK] Constraint violation at {'D1': 1, 'D2': 1}
Step 3: Trying D2 = Afternoon (2)
-> Valid partial assignment: {'D1': 1, 'D2': 2}
Step 4: Trying D3 = Morning (1)
-> [PRUNED / BACKTRACK] Constraint violation at {'D1': 1, 'D2': 2, 'D3': 1}
Step 5: Trying D3 = Afternoon (2)
-> [PRUNED / BACKTRACK] Constraint violation at {'D1': 1, 'D2': 2, 'D3': 2}
Step 6: Trying D3 = Night (3)
-> Valid partial assignment: {'D1': 1, 'D2': 2, 'D3': 3}

```

[GOAL REACHED] Valid Assignment Found: {'D1': 1, 'D2': 2, 'D3': 3}

```

=====
    FINAL VALID SCHEDULE
=====
D1 -> Morning
D2 -> Afternoon
D3 -> Night
=====

```

Process finished with exit code 0

QUESTION 2

1. Problem Setup

- **Grid Size:** 5×5 (Coordinates (r, c) where $0 \leq r, c \leq 4$)
- **Start (\$S\$):** $(0, 0)$
- **Goal (\$G\$):** $(4, 4)$
- **Obstacles (\$X\$):** $(1, 1), (1, 2), (2, 1), (3, 3)$
- **Move Cost (\$g(n)\$):** $\$1$ per step (Up, Down, Left, Right)
- **Heuristic (\$h(n)\$):** Manhattan Distance to Goal $(4, 4)$

$$h(r, c) = |r - 4| + |c - 4|$$
- **Evaluation Function:** $f(n) = g(n) + h(n)$

2. A* Step-by-Step Node Expansion & Priority Queue Updates

Step	Current Node (r,c)	g(n)	h(n)	f(n)	Expanded Neighbors & f(n)	Priority Queue (Open List)
0	Start $(0,0)$	0	8	8	Down $(1,0)$, Right $(0,1)$	$[(0,0): f=8]$
1	$(0,0)$	0	8	8	$(1,0): g=1, h=7, f=8$ $(0,1): g=1, h=7, f=8$	$[(1,0): f=8, (0,1): f=8]$
2	$(1,0)$	1	7	8	$(2,0): g=2, h=6, f=8$	$[(2,0): f=8, (0,1): f=8]$

					<i>(1,1) is Obstacle</i>	
3	\$(2,0)\$	2	6	8	\$(3,0)\$: g=3, h=5, f=8\$ <i>(2,1) is Obstacle</i>	[(3,0): f=8, (0,1): f=8]
4	\$(3,0)\$	3	5	8	\$(4,0)\$: g=4, h=4, f=8\$ \$(3,1)\$: g=4, h=4, f=8\$	[(4,0): f=8, (3,1): f=8, (0,1): f=8]
5	\$(3,1)\$	4	4	8	\$(3,2)\$: g=5, h=3, f=8\$	[(3,2): f=8, (4,0): f=8, (0,1): f=8]
6	\$(3,2)\$	5	3	8	\$(4,2)\$: g=6, h=2, f=8\$ <i>(3,3) is Obstacle</i>	[(4,2): f=8, (4,0): f=8, (0,1): f=8]

7	$(4,2)$	6	2	8	$(4,3)$: g=7, h=1, f=8	$[(4,3): f=8, (4,0): f=8, (0,1): f=8]$
8	$(4,3)$	7	1	8	$(4,4)$: g=8, h=0, f=8	$[(4,4): f=8, (4,0): f=8, (0,1): f=8]$
9	$(4,4)$	8	0	8	GOAL REACHED!	—

3. Optimal Path & Cost

- **Optimal Path:**
 $(0,0) \rightarrow (1,0) \rightarrow (2,0) \rightarrow (3,0) \rightarrow (3,1) \rightarrow (3,2) \rightarrow (4,2) \rightarrow (4,3) \rightarrow (4,4)$
- **Optimal Path Cost ($g(n)$): 8**

```
import heapq
```

```
# 5x5 Grid representation: 0 = Free cell, 1 = Obstacle
```

```
# Grid dimensions
```

```
ROWS, COLS = 5, 5
```

```
# Start and Goal positions
```

```
START = (0, 0)
```

```
GOAL = (4, 4)
```

```
# Obstacle coordinates
```

```
OBSTACLES = {(1, 1), (1, 2), (2, 1), (3, 3)}
```

```
# Movement directions: Up, Down, Left, Right
```

```
DIRECTIONS = [(-1, 0), (1, 0), (0, -1), (0, 1)]
```

```
def manhattan_distance(p1, p2):
```

```
    """Calculates Manhattan Distance heuristic h(n)."""
```

```
    return abs(p1[0] - p2[0]) + abs(p1[1] - p2[1])
```

```
def a_star_search(start, goal):
```

```
    """Executes A* search algorithm and prints priority queue updates."""
```

```
    # Priority Queue stores tuples: (f_score, counter, current_node)
```

```
    # Counter handles tie-breaking in priority queue
```

```
    counter = 0
```

```
    pq = []
```

```
    g_score = {start: 0}
```

```
    f_score = {start: manhattan_distance(start, goal)}
```

```
    heapq.heappush(pq, (f_score[start], counter, start))
```

```
    parent = {}
```

```
visited = set()
```

```
step = 0
```

```
print("=" * 65)
```

```
print("          A* SEARCH STEP-BY-STEP TRACE          ")
```

```
print("=" * 65)
```

```
while pq:
```

```
    current_f, _, current = heapq.heappop(pq)
```

```
    if current in visited:
```

```
        continue
```

```
    visited.add(current)
```

```
    step += 1
```

```
    print(f"\nStep {step}:")
```

```
    print(f"  Popped Node: {current} | g = {g_score[current]}, h =  
{manhattan_distance(current, goal)}, f = {current_f}")
```

```
    # Check for Goal
```

```
    if current == goal:
```

```
        print("\n>>> GOAL REACHED! <<<")
```

```
        break
```


Explore neighbors

for dr, dc in DIRECTIONS:

neighbor = (current[0] + dr, current[1] + dc)

r, c = neighbor

Check grid bounds and obstacles

if 0 <= r < ROWS and 0 <= c < COLS and neighbor not in OBSTACLES:

tentative_g = g_score[current] + 1

if neighbor not in g_score or tentative_g < g_score[neighbor]:

parent[neighbor] = current

g_score[neighbor] = tentative_g

h = manhattan_distance(neighbor, goal)

f = tentative_g + h

f_score[neighbor] = f

counter += 1

heapq.heappush(pq, (f, counter, neighbor))

print(f" Expanded Neighbor {neighbor}: g={tentative_g}, h={h}, f={f}")

Reconstruct optimal path

path = []

curr = goal

while curr in parent:

```
    path.append(curr)

    curr = parent[curr]

path.append(start)

path.reverse()


return path, g_score[goal]
```

```
def print_grid_visualization(path):

    """Displays visual grid in terminal."""

    print("\n" + "=" * 35)

    print("    GRID VISUALIZATION    ")

    print("=" * 35)

    path_set = set(path)

    for r in range(ROWS):

        row_str = ""

        for c in range(COLS):

            pos = (r, c)

            if pos == START:

                row_str += " S "

            elif pos == GOAL:

                row_str += " G "

            elif pos in OBSTACLES:

                row_str += " X "
```

```

        elif pos in path_set:

            row_str += " * "

        else:

            row_str += " . "

    print(row_str)

print("=" * 35)

print(" Legend: S=Start, G=Goal, X=Obstacle, *=Path, .=Empty\n")

```

```

if __name__ == "__main__":

    optimal_path, path_cost = a_star_search(START, GOAL)


    print("\n" + "=" * 65)

    print(f"Optimal Path: {optimal_path}")

    print(f"Total Path Cost: {path_cost}")

    print("=" * 65)


    print_grid_visualization(optimal_path)

```

```

=====

```

A* SEARCH STEP-BY-STEP TRACE

```

=====

```

Step 1:

Popped Node: (0, 0) | g = 0, h = 8, f = 8

Expanded Neighbor (1, 0): g=1, h=7, f=8

Expanded Neighbor (0, 1): g=1, h=7, f=8

Step 2:

Popped Node: (1, 0) | g = 1, h = 7, f = 8

Expanded Neighbor (2, 0): g=2, h=6, f=8

Step 3:

Popped Node: (0, 1) | g = 1, h = 7, f = 8

Expanded Neighbor (0, 2): g=2, h=6, f=8

Step 4:

Popped Node: (2, 0) | g = 2, h = 6, f = 8

Expanded Neighbor (3, 0): g=3, h=5, f=8

Step 5:

Popped Node: (0, 2) | g = 2, h = 6, f = 8

Expanded Neighbor (0, 3): g=3, h=5, f=8

Step 6:

Popped Node: (3, 0) | g = 3, h = 5, f = 8

Expanded Neighbor (4, 0): g=4, h=4, f=8

Expanded Neighbor (3, 1): g=4, h=4, f=8

Step 7:

Popped Node: (0, 3) | g = 3, h = 5, f = 8

Expanded Neighbor (1, 3): g=4, h=4, f=8

Expanded Neighbor (0, 4): g=4, h=4, f=8

Step 8:

Popped Node: (4, 0) | g = 4, h = 4, f = 8

Expanded Neighbor (4, 1): g=5, h=3, f=8

Step 9:

Popped Node: (3, 1) | g = 4, h = 4, f = 8

Expanded Neighbor (3, 2): g=5, h=3, f=8

Step 10:

Popped Node: (1, 3) | g = 4, h = 4, f = 8

Expanded Neighbor (2, 3): g=5, h=3, f=8

Expanded Neighbor (1, 4): g=5, h=3, f=8

Step 11:

Popped Node: (0, 4) | g = 4, h = 4, f = 8

Step 12:

Popped Node: (4, 1) | g = 5, h = 3, f = 8

Expanded Neighbor (4, 2): g=6, h=2, f=8

Step 13:

Popped Node: (3, 2) | g = 5, h = 3, f = 8

Expanded Neighbor (2, 2): g=6, h=4, f=10

Step 14:

Popped Node: (2, 3) | g = 5, h = 3, f = 8

Expanded Neighbor (2, 4): g=6, h=2, f=8

Step 15:

Popped Node: (1, 4) | g = 5, h = 3, f = 8

Step 16:

Popped Node: (4, 2) | g = 6, h = 2, f = 8

Expanded Neighbor (4, 3): g=7, h=1, f=8

Step 17:

Popped Node: (2, 4) | g = 6, h = 2, f = 8

Expanded Neighbor (3, 4): g=7, h=1, f=8

Step 18:

Popped Node: (4, 3) | g = 7, h = 1, f = 8

Expanded Neighbor (4, 4): g=8, h=0, f=8

Step 19:

Popped Node: (3, 4) | g = 7, h = 1, f = 8

Step 20:

Popped Node: (4, 4) | g = 8, h = 0, f = 8

>>> GOAL REACHED! <<<

=====

Optimal Path: [(0, 0), (1, 0), (2, 0), (3, 0), (4, 0), (4, 1), (4, 2), (4, 3), (4, 4)]

Total Path Cost: 8

=====

=====

GRID VISUALIZATION

=====

```
S . . . .  
* X X . .  
* X . . .  
* . . X .  
* * * * G
```

=====

Legend: S=Start, G=Goal, X=Obstacle, *=Path, .=Empty

Process finished with exit code 0

QUESTION 3

Task A: Constraint Impact Analysis

1. 4-Directional Movement: Limits branching factor to $b \leq 4$.
2. Blocked Cells: Requires dynamic obstacle detection before moving.
3. Adjacent Sensing (Partially Observable): Forces an Online Search approach (sensing, updating map, and moving step-by-step) because the full grid layout is unknown.
4. Variable Step Costs: Normal moves cost 1; Risky moves cost $1 + 2 = \mathbf{3}$. Algorithms must track path cost $g(n)$, not just step count.
5. Cost Minimization & Safety: Evaluates paths to prefer a longer clear route over a costly/hazardous shortcut (e.g., $4 \times 1 = 4$ is better than $2 \times 3 = 6$).

Task B & D: Algorithm Approach & Justification

Approach: Online Learning Real-Time A* (LRTA*)

1. Sense: Read adjacent 4 cells using local sensors.
2. Update: Mark discovered obstacles and set cell costs (\$1 for clear, \$3 for risky).
3. Select Move: Choose neighboring cell s' that minimizes:
$$f(s') = \text{cost}(s, s') + h(s')$$
(where $h(s')$ is the Manhattan Distance to Goal G)
4. Execute & Learn: Update local heuristic $h(s) \leftarrow f(s')$ to avoid loops/dead ends, then step forward.

Why it's best: Offline algorithms (standard A*/UCS) fail because the map is initially unknown. LRTA* handles partial observability, avoids hazardous high-cost cells, and guarantees escaping dead-ends.

Task C: AI Concepts Summary

- PEAS: Performance (minimize cost, rescue survivor safely) | Environment (grid with obstacles & risk zones) | Actuators (directional motors) | Sensors (adjacent proximity & hazard detectors).

- **Rationality:** Always picks the lowest expected cost path given current map knowledge.
- **Environment Type:** Partially Observable, Deterministic, Sequential, Static, Discrete, Single-Agent.

```
import math
```

```
# Grid Configuration
```

```
GRID_SIZE = 5
```

```
START = (0, 0)
```

```
GOAL = (4, 4)
```

```
# Actual Environment (Unknown to robot initially)
```

```
REAL_OBSTACLES = {(1, 1), (1, 2), (2, 1), (3, 3)}
```

```
REAL_RISKY_ZONES = {(2, 2), (3, 2)} # Cost = 3
```

```
DIRECTIONS = [(-1, 0), (1, 0), (0, -1), (0, 1)] # Up, Down, Left, Right
```

```
def manhattan_distance(pos):
```

```
    """Heuristic h(n): Manhattan distance to Goal."""
```

```
    return abs(pos[0] - GOAL[0]) + abs(pos[1] - GOAL[1])
```

```
class OnlineRescueRobot:
```

```
    def __init__(self, start, goal):
```

```

self.current_pos = start

self.goal = goal

self.known_obstacles = set()

self.known_risky = set()

# Heuristic lookup table h(s) initialized with Manhattan distance

self.h_table = {}

def get_h(self, pos):

    if pos not in self.h_table:

        self.h_table[pos] = manhattan_distance(pos)

    return self.h_table[pos]

def sense_adjacent(self):

    """Senses adjacent cells and updates internal map knowledge."""

    r, c = self.current_pos

    for dr, dc in DIRECTIONS:

        adj = (r + dr, c + dc)

        if 0 <= adj[0] < GRID_SIZE and 0 <= adj[1] < GRID_SIZE:

            if adj in REAL_OBSTACLES:

                self.known_obstacles.add(adj)

            elif adj in REAL_RISKY_ZONES:

                self.known_risky.add(adj)

def get_step_cost(self, neighbor):

```

```
"""Returns step cost: 3 for risky cells, 1 for normal clear cells."""
```

```
return 3 if neighbor in self.known_risky else 1
```

```
def step(self):
```

```
    """Executes one online search step using LRTA* decision rule."""
```

```
    self.sense_adjacent()
```

```
    best_move = None
```

```
    min_f = math.inf
```

```
    r, c = self.current_pos
```

```
    # Evaluate valid unblocked adjacent cells
```

```
    for dr, dc in DIRECTIONS:
```

```
        neighbor = (r + dr, c + dc)
```

```
        if 0 <= neighbor[0] < GRID_SIZE and 0 <= neighbor[1] < GRID_SIZE:
```

```
            if neighbor not in self.known_obstacles:
```

```
                cost = self.get_step_cost(neighbor)
```

```
                f_val = cost + self.get_h(neighbor)
```

```
                if f_val < min_f:
```

```
                    min_f = f_val
```

```
                    best_move = neighbor
```

```
    # Update local heuristic (Learning step to prevent dead-end loops)
```

```
self.h_table[self.current_pos] = min_f

# Move robot

self.current_pos = best_move

return best_move, self.get_step_cost(best_move)
```

```
def run_simulation():
```

```
    robot = OnlineRescueRobot(START, GOAL)
```

```
    total_cost = 0
```

```
    path = [START]
```

```
    print("=" * 55)
```

```
    print("    ONLINE RESCUE ROBOT NAVIGATION SIMULATION    ")
```

```
    print("=" * 55)
```

```
    step = 0
```

```
    while robot.current_pos != GOAL:
```

```
        step += 1
```

```
        pos, move_cost = robot.step()
```

```
        total_cost += move_cost
```

```
        path.append(pos)
```

```
        cell_type = "Risky (Cost=3)" if pos in REAL_RISKY_ZONES else "Clear (Cost=1)"
```

```
        print(f"Step {step:2d}: Moved to {pos} | Cell: {cell_type} | Path Cost: {total_cost}")
```

```
print("\n" + "=" * 55)

print(">>> SURVIVOR LOCATED SUCCESSFULLY! <<<")

print(f"Optimal Path Taken: {path}")

print(f"Total Path Cost: {total_cost}")

print("=" * 55)
```

```
if __name__ == "__main__":

    run_simulation()
```

=====

ONLINE RESCUE ROBOT NAVIGATION SIMULATION

=====

Step 1: Moved to (1, 0) | Cell: Clear (Cost=1) | Path Cost: 1

Step 2: Moved to (2, 0) | Cell: Clear (Cost=1) | Path Cost: 2

Step 3: Moved to (3, 0) | Cell: Clear (Cost=1) | Path Cost: 3

Step 4: Moved to (4, 0) | Cell: Clear (Cost=1) | Path Cost: 4

Step 5: Moved to (4, 1) | Cell: Clear (Cost=1) | Path Cost: 5

Step 6: Moved to (4, 2) | Cell: Clear (Cost=1) | Path Cost: 6

Step 7: Moved to (4, 3) | Cell: Clear (Cost=1) | Path Cost: 7

Step 8: Moved to (4, 4) | Cell: Clear (Cost=1) | Path Cost: 8

=====

>>> SURVIVOR LOCATED SUCCESSFULLY! <<<

Optimal Path Taken: [(0, 0), (1, 0), (2, 0), (3, 0), (4, 0), (4, 1), (4, 2), (4, 3), (4, 4)]

Total Path Cost: 8

=====

Process finished with exit code 0